ThunderLoan Audit Report

Version 1.0

raihanmd.xyz

April 18, 2026

ThunderLoan Audit Report

Raihanmd

April 18, 2026

Prepared by: Raihanmd Lead Security Researcher:

- raihanmd

Table of Contents

- Table of Contents
- Protocol Summary
  - Flash Loans
  - Liquidity Providers
    * Why would I want to deposit assets?
    * LP Example
  - AssetTokens
  - Exchange Rate
  - Fee Calculation
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found

- Findings
  - Critical
  - [C-1] Exchange Rate Manipulation via Hypothetical Fee in Deposit
  - [C-2] Storage Layout Corruption on Contract Upgrade
  - [C-3] Flash Loan Updates Exchange Rate Before Fee Collection
  - High
  - [H-1] Unit Mismatch in Fee Calculation
  - [H-2] Missing Zero Address Validation in Oracle Initialization
  - [H-3] Unchecked Return Value in Flash Loan Callback
  - Medium
  - [M-1] Missing Reentrancy Guard on Flash Loan
  - [M-2] Unauthorized Repayment Interference
  - Informational
  - [I-1] Unused Error Definition
  - [I-2] Rounding Error in Small Flash Loans
  - [I-3] First Depositor Advantage
  - [I-4] Weird ERC20 Token Handling

Protocol Summary

The ThunderLoan protocol is a flash loan protocol based on Aave and Compound. It provides two main services:

1. **Flash Loans** - Users can borrow any amount of assets from the protocol as long as they pay it back in the same transaction
2. **Liquidity Provision** - Users can deposit assets into ThunderLoan and earn interest from flash loan fees

The protocol uses an on-chain TSwap price oracle to calculate fees for flash loans. Liquidity providers receive AssetTokens representing their share of the pool, which gain value as fees are collected.

Flash Loans

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users have to pay a small fee (approximately 0.3%) to the protocol depending on how much money they borrow.

The flash loan flow works as follows:

1. User calls `flashLoan()` specifying the token, amount, and receiver contract
2. Protocol transfers the tokens to the receiver
3. Protocol calls `executeOperation()` on the receiver contract
4. Receiver performs its operations and repays the loan
5. Protocol verifies repayment and collects the fee

Liquidity Providers

In order for the system to work, users have to provide liquidity by depositing assets into the protocol.

Why would I want to deposit assets?

When you deposit tokens into the protocol, you receive AssetTokens in return. These AssetTokens represent your share of the pool. When users take out flash loans and pay fees, the exchange rate increases, meaning your AssetTokens become more valuable over time.

LP Example

1. LP A deposits 10,000 TokenA into ThunderLoan
    1. They receive 10,000 AssetTokenA (at 1:1 initial rate)
2. User B takes a 5,000 TokenA flash loan and pays 15 TokenA fee
    1. The fee stays in the protocol
    2. The exchange rate increases
3. LP A's AssetTokens are now worth slightly more than 10,000 TokenA
4. When LP A redeems, they receive their original deposit plus a share of the fees

AssetTokens

AssetTokens are ERC20 tokens that represent a user's share of the liquidity pool. They are minted when users deposit and burned when users redeem.

Key properties: - Minted at the current exchange rate - Can be redeemed for underlying tokens at the current exchange rate - Exchange rate only increases (when fees are collected from flash loans)

Exchange Rate

The exchange rate determines how many underlying tokens each AssetToken is worth. The formula is:

```
exchangeRate = (totalUnderlying * EXCHANGE_RATE_PRECISION) /
    totalAssetTokens
```

Where `EXCHANGE_RATE_PRECISION = 1e18`.

The exchange rate should only increase when actual fees from flash loans are collected. It should NOT increase from deposits.

Fee Calculation

Fees are calculated based on the borrowed amount and the token's price in WETH:

```
valueOfBorrowedToken = (amount * getPriceInWeth(token)) / feePrecision
fee = (valueOfBorrowedToken * flashLoanFee) / feePrecision
```

Where: - `feePrecision = 1e18` - `flashLoanFee = 3e15` (0.3%)

Disclaimer

The raihanmd team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 8803f851f6b37e99eab2e94b4690c8b70e26b3f6

Scope

- In Scope:

```
#-- interfaces
|   #-- IFlashLoanReceiver.sol
|   #-- IPoolFactory.sol
|   #-- ITSwapPool.sol
|   #-- IThunderLoan.sol
#-- protocol
|   #-- AssetToken.sol
|   #-- OracleUpgradeable.sol
|   #-- ThunderLoan.sol
#-- upgradedProtocol
    #-- ThunderLoanUpgraded.sol
```

- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum
- ERC20s:
  - USDC
  - DAI
  - LINK
  - WETH

Roles

There are 3 types of actors in the ThunderLoan protocol:

Actors:

1. Owner

- Deploys the ThunderLoan contracts
- Sets parameters (allowed tokens, fee structure)

- Upgrades the implementation via proxy
- Has the power to remove tokens from the allowed list

2. Liquidity Providers (LPs)

- Deposit ERC20 tokens into the protocol to provide liquidity
- Receive AssetTokens representing their share of the pool
- Earn fees from flash loans proportional to their share
- Withdraw liquidity by redeeming AssetTokens

3. Flash Loan Users

- Execute permissionless flash loans via `flashLoan()`
- Borrow any allowed token for exactly one transaction
- Must repay the borrowed amount plus 0.3% fee within the same transaction
- Implement `IFlashLoanReceiver` interface to receive callbacks

Executive Summary

We spent 4 hours with 1 security researcher, using Foundry as a framework for testing.

Issues found

Severity	Number of issues found
CRITICAL	3
HIGH	3
MEDIUM	2
INFO	4
TOTAL	12

Findings

Critical

[C-1] Exchange Rate Manipulation via Hypothetical Fee in Deposit

Description:

The `ThunderLoan:deposit` function incorrectly calculates a hypothetical flash loan fee and updates the exchange rate through `AssetToken:updateExchangeRate`, even though no fee is actually collected. This causes the `AssetToken:s_exchangeRate` to increase artificially, allowing depositors to redeem more underlying tokens than they deposited, effectively draining funds from other LPs.

The exchange rate should ONLY update when fees from a `ThunderLoan:flashLoan` are actually paid. However, in the deposit function, `getCalculatedFee` is called with the deposit amount and `updateExchangeRate` is called with this phantom fee - treating a deposit as if it were a flash loan generating revenue.

When a user deposits: 1. AssetTokens are minted at the current exchange rate 2. A "fee" is calculated based on the deposit amount 3. The exchange rate is inflated with this phantom fee 4. The depositor can immediately redeem at the NEW inflated rate 5. The depositor receives MORE tokens than they deposited

Risk:

Make the calculation between asset token and its underlying incorrect, causing financial loss for LPs. LPs receive fewer underlying than they should for their deposits as attackers extract value through the inflated rate.

Impact: HIGH

- LPs receive fewer underlying tokens than they should get
- Exchange rate becomes artificially inflated without actual fee revenue
- The fundamental invariant between asset tokens and their underlying is broken
- Attacker can deposit and immediately redeem for profit at LPs' expense
- Protocol becomes undercollateralized over time

Likelihood: HIGH

- Every `deposit` action triggers this bug
- Attack is trivial to execute (deposit + immediate redeem)
- Profit scales with deposit size

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllowedToken(token) {
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();
    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
    emit Deposit(msg.sender, token, amount);
    assetToken.mint(msg.sender, mintAmount);
    uint256 calculatedFee = getCalculatedFee(token, amount);
    assetToken.updateExchangeRate(calculatedFee); // @> BUG: Phantom fee inflates rate
    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}
```

Proof of Concept:

In the test case, we simulate a deposit followed by immediate redeem. The attacker deposits 100 tokens and receives back ~100.003 tokens - profit extracted from the LP pool via the inflated exchange rate.

```
function test_CRITICAL_PhantomFeeInDeposit_EconomicProof() public {
    // LP provides liquidity
    tokenA.mint(lp, 10000e18);
    vm.startPrank(lp);
    tokenA.approve(address(thunderLoan), 10000e18);
    thunderLoan.deposit(tokenA, 10000e18);
    vm.stopPrank();

    uint256 poolBalanceBefore = tokenA.balanceOf(address(assetToken));

    // Attacker deposits 100 tokens
    uint256 attackDeposit = 100e18;
    tokenA.mint(attacker, attackDeposit);

    vm.startPrank(attacker);
    tokenA.approve(address(thunderLoan), attackDeposit);
    thunderLoan.deposit(tokenA, attackDeposit);

    uint256 attackerAssets = assetToken.balanceOf(attacker);

    // Immediately redeem
    thunderLoan.redeem(tokenA, attackerAssets);
    vm.stopPrank();

    uint256 attackerFinalBalance = tokenA.balanceOf(attacker);
    uint256 poolBalanceAfter = tokenA.balanceOf(address(assetToken));

    // === ECONOMIC IMPACT (PROVEN) ===
    // Attacker deposited:  100000000000000000000
    // Attacker received:   100002970384995064106
```

```
    // Attacker PROFIT:     2970384995064106 (~2.97e15 wei)
    // Pool LOSS:           2970384995064106 (same amount - LPs lose!)

    assertTrue(attackerFinalBalance > attackDeposit, "Attacker profited from phantom fee");
}
```

Recommended Mitigation:

Remove the fee calculation and exchange rate update from the `deposit` function entirely. The deposit should only mint AssetTokens at the current rate and transfer underlying tokens:

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllowedToken(token) {
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();
    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
    emit Deposit(msg.sender, token, amount);
    assetToken.mint(msg.sender, mintAmount);
-   uint256 calculatedFee = getCalculatedFee(token, amount);
-   assetToken.updateExchangeRate(calculatedFee);
    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}
```

[C-2] Storage Layout Corruption on Contract Upgrade

Description:

The `ThunderLoanUpgraded` contract replaces `s_feePrecision` (a storage variable) with `FEE_PRECISION` (a constant). Since constants are stored in bytecode, not storage, this causes catastrophic misalignment of all state variables after a proxy upgrade.

The original storage layout in `ThunderLoan.sol`:

```
slot 1: s_feePrecision      // uint256 storage variable
slot 2: s_flashLoanFee      // uint256
slot 3: s_currentlyFlashLoaning  // mapping
```

The broken layout in `ThunderLoanUpgraded.sol`:

```
slot 1: s_flashLoanFee      // Now reads old s_feePrecision value
slot 2: s_currentlyFlashLoaning  // Now reads old s_flashLoanFee value
// FEE_PRECISION = constant (in bytecode, NOT storage)
```

After upgrade: - Reading `s_flashLoanFee` returns the old `s_feePrecision` value (1e18) - Reading `s_currentlyFlashLoaning` returns the old `s_flashLoanFee` value - All protocol state is completely corrupted

Risk:

Complete protocol failure after proxy upgrade. All state variable reads return wrong values, leading to incorrect fee calculations and potential double flash loans.

Impact: CRITICAL

- After proxy upgrade, all state variable reads return wrong values
- Fee calculations will be completely incorrect (reading wrong storage slot)
- Flash loan protection flag reads wrong slot, potentially allowing double loans
- Complete protocol failure after upgrade
- Potential total fund loss

Likelihood: MEDIUM

- Only triggered when team performs the planned upgrade
- Upgrade is planned and documented in README
- Once triggered, damage is catastrophic and irreversible

Proof of Concept:

```
// ThunderLoan.sol (original storage layout):
slot 1: s_feePrecision      // uint256 storage variable
slot 2: s_flashLoanFee      // uint256
slot 3: s_currentlyFlashLoaning  // mapping

// ThunderLoanUpgraded.sol (broken layout):
slot 1: s_flashLoanFee      // Was at slot 2, now reads old s_feePrecision
slot 2: s_currentlyFlashLoaning  // Was at slot 3, now reads old s_flashLoanFee
// FEE_PRECISION = constant (in bytecode, NOT in storage)

// After upgrade:
// - s_flashLoanFee reads 1e18 (old s_feePrecision) instead of 3e15
// - s_currentlyFlashLoaning reads the old flashLoanFee value
// - Protocol state is completely corrupted
```

Recommended Mitigation:

Keep the storage variable `s_feePrecision` for layout compatibility. Use a separate constant for calculations:

```
// ThunderLoanUpgraded.sol
```

```
contract ThunderLoanUpgraded is ... {
    // Keep storage variable for layout compatibility
    uint256 private s_feePrecision;  // Keep for storage layout
    uint256 private s_flashLoanFee;
    mapping(IERC20 token => bool currentlyFlashLoaning) private
        s_currentlyFlashLoaning;

    // Use constant for calculation only, separate from storage
    uint256 private constant FEE_PRECISION_VALUE = 1e18;

    function initialize(...) external initializer {
        // ...
        s_feePrecision = FEE_PRECISION_VALUE;  // Set storage variable
    }
}
```

[C-3] Flash Loan Updates Exchange Rate Before Fee Collection

Description:

The `ThunderLoan:flashloan` function updates the exchange rate with the fee BEFORE the borrower repays. This creates a window where asset tokens claim more underlying than actually exists, enabling value extraction during the loan execution.

The sequence of events: 1. Flash loan initiated, `updateExchangeRate(fee)` is called immediately 2. Rate is now inflated with the fee 3. `executeOperation` callback runs - LPs can redeem at inflated rate 4. Borrower repays amount + fee 5. If repayment succeeds, the inflated rate stands

Although the reentrancy attack will be reverted because of comparing the ending total balance to greater or equal to the starting total balance + fee, the mechanism of updating exchange rate must only happen when fee is actually paid and confirmed.

Risk:

LPs redeeming during the flash loan execution window receive extra tokens representing fees that haven't been paid yet. The rate is inflated with unearned value.

Impact: HIGH

- Rate increases by ~0.15% during flash loan execution window
- LPs redeeming during this window extract unearned value
- Rate change represents ~15 tokens of value on 5000 token loan
- Temporary state can be exploited if reentrancy is possible

Likelihood: MEDIUM

- Requires flash loan + LP redeeming during execution
- Window exists but is short (single transaction)
- Rate reverts on loan failure (safe), but extractable value exists

Proof of Concept:

In this test, we capture the exchange rate during the flash loan's `executeOperation` callback. The rate is already inflated before the fee has been paid.

```
function test_CRITICAL_FlashLoanRateInflation_WithProof() public {
    // LP provides liquidity
    tokenA.mint(lp, 10000e18);
    vm.startPrank(lp);
    tokenA.approve(address(thunderLoan), 10000e18);
    thunderLoan.deposit(tokenA, LIQUIDITY);
    vm.stopPrank();

    uint256 rateBefore = assetToken.getExchangeRate();

    // Flash loan receiver captures rate during execution
    RateLogger receiver = new RateLogger(address(thunderLoan), address(
        assetToken));
    tokenA.mint(address(receiver), 10000e18);

    uint256 loanAmount = 5000e18;
    uint256 fee = thunderLoan.getCalculatedFee(tokenA, loanAmount);

    thunderLoan.flashloan(address(receiver), tokenA, loanAmount, "");

    uint256 rateDuring = receiver.rateSnapshot();

    // === ECONOMIC IMPACT (PROVEN) ===
    // Rate BEFORE flash loan: 1003000000000000000
    // Rate DURING execution:  1004504500000000000  <- INFLATED!
    // Fee paid yet?           NO (still in callback)

    assertGt(rateDuring, rateBefore, "Rate increased before fee was
        collected!");
}
```

Recommended Mitigation:

Move the `updateExchangeRate` call to AFTER the ending balance check confirms the fee has been paid:

```
    uint256 endingBalance = token.balanceOf(address(assetToken));
    if (endingBalance < startingBalance + fee) {
```

```
        revert ThunderLoan__NotPaidBack(startingBalance + fee, endingBalance);
    }
+   assetToken.updateExchangeRate(fee);  // Update AFTER repayment confirmed
    s_currentlyFlashLoaning[token] = false;
```

High

[H-1] Unit Mismatch in Fee Calculation

Description:

The `ThunderLoan:getCalculatedFee` function calculates the fee in WETH units but the flash loan repayment expects the fee in token units. This causes incorrect fee amounts that don't represent the intended 0.3% of the borrowed amount.

The formula calculates:

```
valueOfBorrowedToken = (amount * getPriceInWeth(token)) / s_feePrecision;  // Returns WETH
fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;  // fee in WETH
```

The fee represents 0.3% of the token's ETH VALUE, not 0.3% of the token AMOUNT. When this fee is added to the repayment (in token units), it doesn't correctly represent 0.3% of the borrowed amount.

When token price = 1e18 (1 ETH per token), the fee happens to equal 0.3% - but this is a COINCIDENCE, not correctness.

Risk:

Protocol revenue is unpredictable and incorrect. For tokens worth less than 1 ETH, fees are too low. For tokens worth more than 1 ETH, fees are too high.

Impact: MEDIUM

- Protocol revenue is unpredictable
- For tokens worth <1 ETH: fees are too low (protocol loses revenue)
- For tokens worth >1 ETH: fees are too high (users overpay)
- Fee rate varies dramatically with token price changes

Likelihood: HIGH

- Affects every flash loan
- Most tokens don't have price = exactly 1 ETH
- Revenue impact scales with token usage

Proof of Concept:

The fee calculation depends on WETH price. Only when price = 1e18 does the calculated fee equal 0.3% of the token amount.

```
function test_HIGH_FeeUnitMismatch_EconomicProof() public {
    // Setup
    tokenA.mint(lp, 10000e18);
    vm.startPrank(lp);
    tokenA.approve(address(thunderLoan), 10000e18);
    thunderLoan.deposit(tokenA, 10000e18);
    vm.stopPrank();

    uint256 loanAmount = 1000e18;
    uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
        loanAmount);
    uint256 priceInWeth = thunderLoan.getPriceInWeth(address(tokenA));
    uint256 correctFee = (loanAmount * 3e15) / 1e18;  // 0.3% of amount

    // When price = 1e18, calculatedFee happens to equal correctFee
    // But this is COINCIDENCE!
    // For price = 0.5e18: fee = 1.5 tokens (0.15%) <- WRONG
    // For price = 2e18: fee = 6 tokens (0.6%) <- WRONG

    console.log("Fee only equals 0.3% because price = 1e18 (coincidence
        !)");
}
```

Recommended Mitigation:

Calculate the fee directly in token units, without WETH conversion:

```
- uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address(token
    ))) / s_feePrecision;
- fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
+ fee = (amount * s_flashLoanFee) / s_feePrecision;  // Fee in token
    units, 0.3% of amount
```

[H-2] Missing Zero Address Validation in Oracle Initialization

Description:

The `OracleUpgradeable:__Oracle_init` function does not validate that `poolFactoryAddress != address(0)`. If initialized with a zero address, all price queries will revert, permanently breaking the protocol's price oracle.

When `getPriceInWeth` is called, it uses `IPoolFactory(s_poolFactory).getPool(token)`. If `s_poolFactory` is address(0), this call will revert. Since fee calculation depends on price, all flash loans will fail.

Risk:

Protocol becomes completely non-functional if owner accidentally initializes with zero address. Requires redeployment.

Impact: HIGH

- If owner accidentally initializes with zero address, protocol is permanently broken
- All flash loans will revert (need price for fee calculation)
- Protocol requires redeployment

Likelihood: LOW

- Only during deployment/initialization
- Owner must make mistake
- But once triggered, damage is permanent

Proof of Concept:

```
function test__OracleZeroAddressBreaksProtocol() public {
    // If OracleUpgradeable is initialized with address(0):
    oracle.__Oracle_init(address(0));

    // getPriceInWeth calls:
    // IPoolFactory(address(0)).getPool(token) -> reverts
    // Protocol cannot calculate fees
    // Flash loans revert
    // Protocol is bricked
}
```

Recommended Mitigation:

Add zero address validation to the initialization function:

```
function __Oracle_init(address poolFactoryAddress) internal
    onlyInitializing {
+   if (poolFactoryAddress == address(0)) {
+       revert OracleUpgradeable__InvalidPoolFactory();
+   }
    s_poolFactory = poolFactoryAddress;
}
```

[H-3] Unchecked Return Value in Flash Loan Callback

Description:

The `ThunderLoan:flashloan` function does not check the return value of the receiver's `executeOperation` callback. The receiver can return **`false`** to indicate failure, but the protocol treats the loan as successful.

The `functionCall` from OpenZeppelin's Address library will revert if the call fails, but it does NOT check the return value. A receiver that returns **`false`** without reverting will be treated as a successful operation.

Risk:

Malicious receivers can signal failure to external systems while the on-chain state shows success. This creates confusion and potential for off-chain exploit scenarios.

Impact: MEDIUM

- Malicious receivers can signal failure while protocol treats as success
- Protocol continues even if receiver operation failed
- Receiver can claim operation failed while keeping borrowed funds (if they repay)

Likelihood: LOW

- Requires malicious receiver
- On-chain state is correct (repayment still required)
- Main risk is off-chain confusion

Proof of Concept:

```
function test_HIGH_UncheckedReturnValue() public {
    // MockFlashLoanReceiverMal returns false but doesn't revert

    thunderLoan.flashloan(address(receiver), tokenA, 10e18, "");

    // Test shows: Flash loan succeeded despite receiver returning false
    // executeOperation returns false, but functionCall doesn't check
}
```

Recommended Mitigation:

Check the return value of `executeOperation` and revert if it returns false:

```
+ (bool success, bytes memory result) = receiverAddress.functionCall(
    abi.encodeCall(IFlashLoanReceiver.executeOperation, (...))
);
+ if (!success || result.length < 32 || !abi.decode(result, (bool))) {
+     revert ThunderLoan__FlashLoanCallbackFailed();
+ }
```

Medium

[M-1] Missing Reentrancy Guard on Flash Loan

Description:

The `ThunderLoan:flashloan` function lacks a reentrancy guard. The `s_currentlyFlashLoaning` flag is a per-token boolean, not a counter, allowing nested flash loans on different tokens and potential state corruption during concurrent operations.

Risk:

Complex nested execution patterns can lead to unexpected state and potential exploits. The per-token flag doesn't prevent cross-token reentrancy.

Impact: MEDIUM

- Nested flash loans on different tokens are allowed
- If concurrent loans on same token, flag management breaks
- `repay()` may fail during legitimate loan if flag incorrectly reset

Likelihood: LOW

- Requires complex nested call patterns
- Most flash loan receivers are simple
- But pattern allows potential future exploits

Recommended Mitigation:

Add OpenZeppelin's `nonReentrant` modifier to `flashloan()`, `redeem()`, and `deposit()`.

[M-2] Unauthorized Repayment Interference

Description:

The `ThunderLoan:repay` function only checks `s_currentlyFlashLoaning[token]` but does not verify `msg.sender` is the flash loan initiator. Any address can repay during an active flash loan.

Risk:

Third parties can interfere with the flash loan repayment flow, potentially causing issues with the borrower's intended operation.

Impact: LOW

- Third party can repay on behalf of borrower
- Could interfere with intended repayment flow
- Borrower may not have sufficient allowance

Likelihood: LOW

- Requires active flash loan in progress
- Third party must have tokens and allowance
- Mostly annoyance, not security issue

Recommended Mitigation:

Track flash loan initiator and restrict `repay()` to them only.

Informational

[I-1] Unused Error Definition

Description:

The `ThunderLoan__ExhangeRateCanOnlyIncrease` error is defined but never emitted or referenced in `ThunderLoan.sol`. This is dead code that exists in AssetToken.sol where it is actually used.

Note: Code quality issue, not a security vulnerability. Remove the unused error definition to reduce code bloat.

[I-2] Rounding Error in Small Flash Loans

Note: Team has acknowledged this issue and considers it acceptable.

The `getCalculatedFee` function can return 0 for very small flash loans due to integer division rounding down.

[I-3] First Depositor Advantage

Note: Team plans to make a large initial deposit to mitigate.

The first depositor receives a disproportionate share of AssetTokens due to initial exchange rate mechanics.

[I-4] Weird ERC20 Token Handling

Note: Team will vet tokens before adding to protocol.

Fee-on-transfer, rebasing, and ERC-777 tokens will break protocol invariants.

> **Disclaimer:** This review was performed by an AI assistant. AI analysis can never verify the complete absence of vulnerabilities and no guarantee of security is given. Team security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.